

Market Data Definition Language (MDDL)

Technical Specification

01 March 2007



<http://www.mddl.org/>

Editors:

Anthony B. Coates, London Market Systems, London, U.K.

James E. Hartley, FISD/SIIA, Denver, U.S.A.

Copyright ©2001-2007 Software & Information Industry Association. All Rights Reserved.

Abstract

Market Data Definition Language (MDDL) is a specification based on the eXtensible Markup Language (XML) standard to enable interchange of data necessary to account for, to analyze, and to trade instruments of the world's financial markets. MDDL seeks, through definition of common terms, to provide a standard vocabulary so market data may be exchanged unambiguously between exchanges, vendors, redistributors, and subscribers. MDDL is designed to facilitate delivery of all data and to increase ease of processing for recipients of this market based financial data.

This document discusses the technical specification of MDDL, the XML W3C Schema (XSD), and sample instance documents.

Acknowledgements

This specification is a cooperative effort of representatives of the financial industry at the behest of the general membership of the [Financial Information Services Division](#) of the [Software & Information Industry Association](#). Those organizations, and the individual contributors, are hereby thanked and commended for contributing staff and energy to this volunteer collective. Inherent competition has been cast aside and common goals have been pursued to bring this industry-wide goal to fruition.

Thanks to Jeremy Sanders who originally proffered the vision and impetus for MDDL, and much appreciation for the guidance, enthusiasm, and support that has been relentlessly effused by Michael Atkin.

Conventions Used in This Document

The following highlighting is used to present technical material in this document:

Complete examples or fragments

The following highlighting is used for non-normative commentary in this document:

Example
<mddl version="2.0-draft" > (Incomplete examples or fragments)
And an explanation of the example.

NOTE: General comments directed to all readers.

All MDDL specific XML elements and attributes will be **bolded in monospace**.

All words, other than elements or attributes, for which MDDL has a specific definition will be *colored and italicized*. The following example paragraph illustrates relevant highlighting:

The main XML element of an instance document, **mddl**, has one required attribute, **version**, containing the explicit version of the W3C Schema used to validate the document. The element **mddl** may then contain a **header** element followed by any of the valid *constructs* for the referenced version of MDDL.

Status of this Document

This is the second complete revision and is available coincident with the public release of Market Data Definition Language Version 2.0-draft in preparation for the *XML for Market Data: Defining the Reference Data Standard* conference in New York, U.S.A. on 28-29 April 2003.

This document may be revised more frequently than the MDDL Schema, which forms the nexus of the specification, and the *Implementation Overview*. Such revisions will be noted within the Change Log section of this document. All modifications to the specification will likely necessitate a re-release of this document.

History of MDDL

Add intro, discuss 1.0 at 5th WFIC. Discuss inception...

Table of Contents

Abstract.....	1
Acknowledgements	2
Conventions Used in This Document	2
Status of this Document	2
History of MDDL.....	2
Table of Contents	2
Change Log.....	2
1 Introduction	2
1.1 Purpose	2
1.2 Terminology	2
1.2.1 Sender.....	2
1.2.2 Recipient.....	2
1.2.3 Property	2
1.2.4 Product	2
1.2.5 Base Type.....	2
1.2.6 Classification.....	2
1.2.7 Construct	2
1.2.8 Groups and Containers	2
1.2.9 Controlled Vocabulary	2
1.2.10 XML Specific Terms.....	2
1.3 Relationship to Other Work.....	2
2 XML Specification & Implementation Concepts	2
3 Related Files.....	2
3.1 Schema – mddl-1.0-final.xsd	2
3.2 DTD – mddl-1.0-final.dtd	2
3.3 Reference – mddlref-1.0-final.xsd	2
3.4 Link – mddlink-1.0-final.xsd.....	2
3.5 Glossary – mddl-1.0-final.saf	2
3.6 Others	2
3.6.1 Schema Documentation – mddl-1.0-final.htm.....	2
3.6.2 XML Schema – xml.xsd	2
3.6.3 Glossary Stylefile – mddl-1.0-final-saf.xsl	2
3.6.4 Glossary in HTML – mddl-1.0-final-saf.htm.....	2
4 MDDL Glossary	2
5 References.....	2

A	Syntax of MDDL Instance Documents	2
A.1	Principal Elements and Attributes	2
A.1.1	Discussion of Attributes	2
A.1.2	Root Element <i>mddl</i>	2
A.1.3	Extension Element <i>other</i>	2
A.1.4	Optional Element <i>header</i>	2
A.1.5	Standard Attributes	2
A.1.6	Use of element <i>instrumentIdentifier</i>	2
A.1.7	Special Controlled Vocabulary <i>indicatorsType</i>	2
A.2	Base Types	2
A.2.1	Base Type <i>Boolean</i>	2
A.2.2	Base Type <i>DateTime</i>	2
A.2.3	Base Type <i>Decimal</i>	2
A.2.4	Base Type <i>Duration</i>	2
A.2.5	Base Type <i>String</i>	2
A.2.6	Base Type <i>Enumeration</i>	2
A.2.7	Base Type <i>URI</i>	2
A.2.8	Adding Types	2
A.3	Classifications	2
A.3.1	Classification <i>Amount</i>	2
A.3.2	Classification <i>DateTime</i>	2
A.3.3	Classification <i>Fee</i>	2
A.3.4	Classification <i>Measure</i>	2
A.3.5	Classification <i>Price</i>	2
A.3.6	Classification <i>Quantity</i>	2
A.3.7	Other Classifications	2
A.4	Constructs	2
A.4.1	Construct <i>snap</i>	2
A.4.2	Construct <i>timeseries</i>	2
A.4.3	Adding Constructs	2
A.5	Products	2
A.5.1	Product Groups as Containers	2
A.5.2	Adding Products	2
A.5.3	List of Products – MDDL 1.0-final	2
A.6	Properties	2
A.6.1	Common Property <i>fraction</i>	2
A.6.2	Common Property <i>multiplier</i>	2
A.6.3	Common Property <i>period</i>	2
A.6.4	Common Property <i>sequence</i>	2
A.6.5	Adding Properties	2
A.6.6	List of Properties – MDDL 1.0-final	2
A.7	Reference Content	2
A.7.1	Referenced Element Trees	2
A.7.2	Referencing Element Trees	2
B	Example MDDL Instance Documents	2
B.1	Example 1 – <i>snap</i> (Mutual Fund)	2
B.2	Example 2 – <i>timeseries</i> (Common Equity)	2
B.3	Example 3 – Basic references (Index)	2
B.4	Example 4 – <i>Controlled Vocabulary Scheme File</i>	2
B.5	Example 5 – Schema Adjunct Framework Glossary	2

Change Log

2007/03/01	JE Hartley	Rebranded for MDDL 3.0-beta
------------	------------	-----------------------------

1 Introduction

Market Data Definition Language (MDDL) is the eXtensible Markup Language (XML) derived specification to enable the interchange of information necessary to account for, to analyze, and to trade financial instruments of the world's markets. MDDL defines a common vocabulary and exchange format to facilitate the interaction between exchanges, vendors, redistributors, and consumers. The common market data terminology embraced by MDDL allows providers to clearly state the nature and origin of market data elements thus removing ambiguity. The straightforward format of MDDL provides a convenient vehicle for exchanging this data amongst XML aware systems.

1.1 Purpose

Included herein is a general outline of the goals of MDDL and details about the MDDL specification. Discussion of the vocabulary of MDDL and related concepts can be found in the *Implementation Overview* available from the sponsoring agency or at mddl.org.

This document is intended for technical implementers or those individuals interested in the specific syntax of MDDL and the rules for construction and processing of instance documents. This document does not seek to educate the reader on the content of MDDL but concentrates on the way XML is used to deliver market data with MDDL.

Covered material includes concepts specific to MDDL as well as XML terminology in general. Included are discussions of special MDDL elements and attributes as well as excerpts and complete examples of instance documents. All included discussions and examples in this document are valid for MDDL Version "2.0-draft". Many may be backward compatible with previous versions (namely "1.0-final").

1.2 Terminology

The MDDL documentation uses various terms that have a specific meaning within the confines of the MDDL specification. These terms are defined here as a way to clarify the intended meaning in reference to MDDL.

Presenting the details of an XML-based specification like MDDL requires the use of various terms unique to that genre. These terms are described in relation to MDDL but also to XML in general.

1.2.1 Sender

A *sender* is any system that generates and provides MDDL instance documents regardless of the industry role of exchange, vendor, redistributor, or consumer. Note that internal MDDL processing within a system may include subsystems that are *senders* as well as *recipients*.

1.2.2 Recipient

A *recipient* is any system that, regardless of industry role, receives or processes MDDL instance documents. These *recipients* of MDDL documents will validate and parse the instances using the rules of the MDDL *version* specified in the instance. *Recipient* systems may process MDDL documents and then provide that data to other *recipients* in MDDL format thus fulfilling the role of *sender* as well.

1.2.3 Property

A *property* is a descriptor or modifier about a piece of data that clarifies the nature of the data. *Properties* hold the real content of MDDL across the full range of market data. MDDL specifies many *properties* that relate to a particular type of financial instrument, and MDDL defines the hierarchical relationship between the *properties*, thus defining a market data *product*. MDDL *properties* may be further enhanced by other *properties* to clarify the implications of the parent *property*. As an example, consider **last** which is *classified* as a price and is further enhanced by the *property* **currency**. See the discussion of *properties* in the appendix for inheritance rules.

Example of *properties*

```
<trade>
  <last>
    <mdDecimal>127.32</mdDecimal>
    <currency>JPY</currency>
  </last>
</trade>
```

The *property* **last** is further clarified by the *property* **currency**.

NOTE: *Properties* can modify other *properties*.

MDDL allows extension of the specification to include proprietary *properties* not defined in the formal specification. The extension element **other**, which is a child of all *properties* and *products*, can be used to extend any *property* or *product*. In addition, MDDL *properties* can be inherited from an ancestor per established inheritance guidelines.

1.2.4 Product

MDDL uses the term *product* to denote the collection of *properties* and the inherent relationship between those *properties* necessary to define a financial instrument or asset, indicator or average, or corporate action or event relevant to market data. Examples of *products* include Mutual Fund Collective Investment Vehicles and Common Equities. The complete list of *products* can be found at mddl.org.

As with *properties*, MDDL permits extension by the addition of *products* within the extension element **other** of the main *constructs* or *containers*.

1.2.5 Base Type

Each MDDL defined *property* has a *base type* defined in the vocabulary. The *base type* specifies the kind of value the *property* can contain in much the same way a type constrains a variable in most programming languages. *Base types* include Boolean, DateTime, Decimal, Duration, String, and URI.

1.2.6 Classification

MDDL uses the term *classification* to denote the fundamental usage of a *property* or element. The *classification* identifies the predefined collection of *properties* that can be used to clarify the parent *property*. The list of predefined *classifications*, including price and quantity, cannot be extended within MDDL without modification to the Schema and DTD. Any need for additional *classifications* should be discussed with the MDDL sponsoring agency. Classifications include Amount, DateTime, Fee, Measure, Price, and Quantity.

1.2.7 Construct

An MDDL *construct* is the collection and grouping of *products* and *properties* necessary to convey a particular reason for providing the market data. For example, the *construct snap* denotes that the included data is a snapshot of a quote – perhaps at end of day.

MDDL 1.0 does not facilitate addition of *constructs* but future revisions will allow proprietary *constructs* as with *properties* or *products*. Please discuss additional *construct* needs with the sponsoring agency.

1.2.8 Groups and Containers

Because of the inheritance rules for *properties*, it is often convenient to have an artificial layer in the hierarchy of an MDDL document where *properties* can be collected that apply to all the descendants of that element. MDDL defines several elements, suffixed with Group, that are intended to be *containers* for just this purpose. These *containers* take on the parameters of their parents but are used solely for the segregation and holding of *properties* for inheritance. Note that any *product* or *property* is actually a *container* although Groups are the only *containers* created specifically for this purpose. Some *properties* are only *containers* in that they do not hold any values other than those attributed to child *properties*.

1.2.9 Controlled Vocabulary

A *controlled vocabulary* is a list of terms intended to constrain the values a particular *property* may contain. All *properties* that use *controlled vocabularies* are of *base type String* with *classification Enumeration* and may take on any values listed in the *controlled vocabulary*. The *controlled vocabularies* are defined and modified external to the MDDL Schema or DTD and can be created as necessary by *senders*.

1.2.10 XML Specific Terms

XML is becoming a more prevalent standard within the computer and data industries and, as such, it has terminology that is unique to that standard.

1.2.10.1 Standard vs. Specification

XML is a metalanguage standard, developed and supported by the World Wide Web Consortium, intended to provide a flexible and adaptable scheme for information identification. MDDL is an XML specification – MDDL uses the XML standard to define the syntax for creating market data related documents.

1.2.10.2 Schema and DTD

MDDL is intended for all *senders* and *recipients* of market data. These organizations have different internal processes and involvement with emerging standards like the W3C XML Schema (heretofore referred to as “Schema”). By supporting both Schema and Document Type Definition (DTD), MDDL allows adoption of MDDL by any organization ready to process XML regardless of that organization’s acceptance of the newer XML Schema standard. While supporting DTD prohibits MDDL from using some of the advanced features of XML Schema, the absence of those capabilities do not detract from the fundamental capabilities and intents of MDDL.

1.2.10.3 Instance Document

While Schemas and DTDs define the XML specification of interest (in our case MDDL), the actual data interchanged between *sender* and *recipient* is formatted in an instance

document based on the rules defined in the specification. An instance document is itself XML and the textual representation is dictated by the XML standard. An instance document is said to be *well-formed* XML if the elements and angle-brackets follow the rules of the XML standard and are correctly started, nested, and terminated *independent* of the governing specification.

An MDDL instance document is only valid if it is well-formed XML, validates against the correct version of the MDDL Schema and DTD, *and* is consistent with the intent of this written specification. Note that it is possible to generate MDDL instance documents that validate against the DTD (or the Schema) but are not valid MDDL because they do not follow the guidelines of this specification. This discrepancy exists because Schema and DTD do not permit the necessary restrictions to encode all MDDL rules. Additionally, the DTD is looser than the Schema such that documents that validate against the Schema will be valid against the DTD, although the reverse may not always be true.

1.2.10.4 Schema Adjunct Framework

MDDL makes use of the Schema Adjunct Framework (SAF) to present the Glossary of MDDL elements and definitions. In essence, the Glossary uses the MDDL namespace to specify additional elements that are not defined in the MDDL Schema or DTD. These elements provide enhancing information about the element including the definition and *classification* of the element. By providing this ancillary information outside the Schema or DTD, updates and additions (including the addition of other languages) can be made independently of the specification. The Glossary and specific implementation of SAF is discussed later in this document.

1.3 Relationship to Other Work

MDDL uses several World Wide Web Consortium (W3C) recommendations including, but not limited to, [XML 1.0](#), [XML Namespaces](#), [XLink](#), and makes specific reference to [XSLT](#). MDDL relies extensively on [W3C XML Schema](#) and DTDs. Please see the [W3C XML Website](#) for references to XML related work. Additionally, MDDL makes use of the [Schema Adjunct Framework](#) for composition of the Glossary.

Other XML specifications in the financial space have direct relationship to MDDL and some integration should be pursued. Targeted specifications for ongoing interoperability discussions include [XBRL](#) (eXtensible Business Reporting Language), [FpML](#) (Financial Products Markup Language), [RIXML](#) (Research Information Exchange Markup Language), and specifications of the [IPTC](#) (International Press Telecommunications Council) including [NewsML](#) and [NITF](#) (News Industry Text Format). Discussions with SWIFT/ISO15022 and ISO Working Group 10 are also ongoing.

2 XML Specification & Implementation Concepts

MDDL was initially construed as an XML specification and, as such, no other framework has been considered. XML is prevalent in many industries outside the financial markets segment and, indeed, has a strong foundation within the segment. The MDDL Technical Committee embraced XML as an accepted standard and developed the specifics of MDDL around that structure.

Early discussions debated the use of Document Type Definition (DTD) or W3C Schema to define the exact syntax of MDDL. Older specifications in the financial market space had adopted DTD initially and some have been converting to Schema. As both formats have advantages, primarily penetration for DTD and functionality for Schema, it was decided to support both methods of representation. Note that a conscious decision was made, without detracting to the basic tenets of MDDL, to use Schema to only represent concepts that can also be represented in DTD.

MDDL is authored in XML Schema in a file called “mddlbase-1.0-final.xsd”. This Schema file is then transformed – with XML StyleSheets ala XSLT – to generate the principal files “mddl-1.0-final.xsd”, “mddlref-1.0-final.xsd”, and “mddl-1.0-final.dtd”.

A complete discussion of the MDDL XML Specification can be found in Appendix A. The structure and format of an MDDL instance document is discussed and numerous examples are provided. More complete MDDL instance document examples are provided in Appendix B as are examples of some of the supporting files.

Every exchange of an MDDL instance document is between a *sender* and a *recipient* regardless of the location of either party or the relationship between them. The *sender* is responsible for formatting the document consistent with the *MDDL Technical Specification* (this document). The *recipient* may choose to validate the document based on the *version* dictated in the root element by using a document validator and by testing for special rules not covered in the XML Schema or DTD. Discrepancies in formatting encountered during this processing should be handled by the parties involved. However, resources at mddl.org, including the discussion bulletin boards, can be used to clarify any issues that are unclear.

MDDL prescribes major collections of *properties* that comprise various financial market data *products*. In most cases, the hierarchical relationship between the *properties* is defined. However, the nature of *property* inheritance permits a very flexible arrangement within an instance document. *Senders* should note that inconsistent use of the hierarchical relationships and inheritance will likely cause undue processing (and frustration) by the *recipient*. Further, the use of proprietary extensions by *senders* should match the general architecture of MDDL to facilitate processing by *recipients*.

Recipients of MDDL documents are encouraged to generalize processing around the *classification* assigned to each element. The *classification* specified in the Glossary indicates the collection of *properties* necessary to fully describe a piece of data. By understanding that *recipients* will search for the requisite *properties*, *senders* can attempt to provide the necessary elements in a convenient fashion. Understandably, no two MDDL providers will generate identically formatted documents but *recipients* can, with minimal extra effort, create a single processor that will accept standard MDDL data from any *sender*.

Many terms used within MDDL do not have unique definitions or can take on multiple meanings depending on the source of the data. These *properties* are defined to have a *controlled vocabulary* associated with them that constrains the values of the *property* to only those terms defined in the vocabulary. The vocabulary has a scheme name assigned to it by the sender, or one from the default, which uniquely identifies the list of values. The default scheme names, for example, point to XML files on the mddl.org website that contain the list of values in a way that can be processed by an XML aware system. Generic XML document validators do not check these special values nor load the *controlled vocabulary* schemes. Error checking of *properties* of this type requires special processing. It is recommended that if this type of error checking is to be performed, the *recipient* should load copies of the schemes into its local system for performance reasons.

For additional details, please see the separate *Implementation Overview* document containing business reasons for MDDL, discussion about the implementation of MDDL, and other general guidelines for generating and processing MDDL documents.

Issues with MDDL formatting, structure, vocabulary, or content should be addressed to the sponsoring organization or the bulletin boards at mddl.org.

3 Related Files

The following describes the function and content of the various files that are provided with each MDDL release. The MDDL 1.0-final Release contains two major ZIP files:

“mddl-1.0-final_20011102.zip” contains all of the documents below INCLUDING the Schema Documentation subdirectory and is approximately 6.1 MB.

“mddl-1.0-final_20011102_nodocs.zip” contains all of the documents below EXCLUDING the Schema Documentation subdirectory and is approximately 68 KB.

3.1 Schema – mddl-1.0-final.xsd

The file “mddl-1.0-final.xsd” contains the XML Schema specification for MDDL. Note that there is no inline documentation discussing the various sections of the Schema. Much of the detailed documentation about the elements is stored in the MDDL Glossary (“mddl-1.0-final.saf”).

3.2 DTD – mddl-1.0-final.dtd

The Document Type Definition is found in “mddl-1.0-final.dtd” and is equivalent to the Schema “mddl-1.0-final.xsd”. Note that an instance may be valid against the DTD but is not a valid MDDL document unless it is also valid against the Schema and follows the guidelines outlined in this document.

3.3 Reference – mddlref-1.0-final.xsd

The file “mddlref-1.0-final.xsd” contains the XML Schema specification for the elements that can be used in the [references](#) section of an MDDL document. Essentially, all MDDL elements can be referenced. This Schema provides the syntax of that use.

3.4 Link – mddlink-1.0-final.xsd

The XLink functionality, used by [references](#) as defined in “mddlref-1.0-final.xsd”, is facilitated by the file “mddlink-1.0-final.xsd”.

3.5 Glossary – mddl-1.0-final.saf

The MDDL Glossary is contained in the file “mddl-1.0-final.saf”. The Glossary contains descriptions about each element including the *base type* and *classification* of the element.

3.6 Others

There are several other files that are considered part of the release although they are ancillary and not required for basic functionality. These files provide extra documentation and supporting features.

3.6.1 Schema Documentation – mddl-1.0-final.htm

The file “mddl-1.0-final.htm” is the front page of the basic Schema Documentation for “mddl-1.0-final.xsd” provided with MDDL. This documentation contains details and descriptions about the use of each of the elements within the current version of MDDL. The full Schema Documentation is provided in the subdirectory “docs”.

3.6.2 XML Schema – xml.xsd

This file, “xml.xsd”, is provided for completeness for namespacing and can also be found online at <http://www.w3.org/2001/xml.xsd>.

3.6.3 Glossary Stylefile – mddl-1.0-final-saf.xsl

This example XML StyleSheet, “mddl-1.0-final-saf.xsl”, is the default XSLT used to render the MDDL Glossary. This file is referenced in “mddl-1.0-final.saf” but is provided as an example only.

3.6.4 Glossary in HTML – mddl-1.0-final-saf.htm

The rendered Glossary (“mddl-1.0-final.saf” transformed by “mddl-1.0-final-saf.xsl”) is provided in the release as a reference.

4 MDDL Glossary

A major goal of the Market Data Definition Language effort is to provide a standardized set of terms that can be understood and shared by all systems that process market data. These terms and corresponding definitions are provided in the MDDL Glossary.

The Glossary contains every *container*, *product*, and *property* used in the current MDDL specification. Each entry in the Glossary has the following additional information:

- MDDL *Classification*
- MDDL *Base Type*
- Definition as provided by Vocabulary Committee
- Labels – Short, Medium, and Long – intended for display applications

Note that the definition and labels may be in other languages in addition to English as identified by the `xml:lang` and `locale` attributes.

No DTD or Schema is currently provided for the Glossary but the format is dictated by the Schema Adjunct Framework as illustrated by the example in Appendix B.

At their discretion, individual *senders* may wish to provide a Glossary for proprietary elements used to extend MDDL.

5 References

XML	http://www.w3c.org/TR/REC-xml and http://www.w3.org/XML/
XML Schema	http://www.w3c.org/TR/xmlschema-0/
- Structures	http://www.w3c.org/TR/xmlschema-1/
- Data Types	http://www.w3c.org/TR/xmlschema-2/
XLink	http://www.w3c.org/TR/xlink/
XPath	http://www.w3c.org/TR/xpath/
XSLT	http://www.w3c.org/TR/xslt/
MathML	http://www.w3c.org/TR/MathML2/
ISO15022	http://www.iso15022.org/
SWIFT	http://www.swift.com/
XBRL	http://www.xbrl.org/
FpML	http://www.fpml.org/
RIXML	http://www.rixml.org/
IPTC	http://www.iptc.org/
NewsML	http://www.newsml.org/
NITF	http://www.nitf.org/
SAF	http://www.extensibility.com/resources/saf.htm

Market Data Definition Language (MDDL)

Appendix A – Syntax of MDDL Instance Documents

Version 1.0-final 02 November 2001



<http://www.mddl.org/>

A Syntax of MDDL Instance Documents

Market Data Definition Language (MDDL) employs a very direct use of XML to represent the inherit hierarchy in the market data *products* and *properties* covered by the specification. The use of XML to describe the market data in the MDDL specification is covered in this appendix.

A.1 Principal Elements and Attributes

The basic framework of MDDL can be divided into two major areas – those elements that describe the real content and those elements and attributes necessary to hold all of the content together in an operable fashion. MDDL relies heavily on elements and the interrelationship between these elements to describe market data. Beyond those necessary to represent the vocabulary, there are only a few elements and attributes that are necessary to convey the significance of the data relationship as defined below.

A.1.1 Discussion of Attributes

The development of MDDL concentrated on the inclusion of explicit extensibility so individual *senders* could add proprietary *products* or *properties* in a way acknowledged and accepted by all parties. While attributes have an appropriate place and are used in MDDL, the hierarchy of the content is represented with XML elements as it is easier to extend the content model of an element than it is to rework extensive numbers of attributes. An extension element, called *other*, is added to all MDDL elements so explicit extensibility can be achieved.

A.1.2 Root Element *mddl*

The root element is the only required element of an MDDL document although a useful MDDL document will contain many more. The root element has a single required attribute called *version*. Although not discussed in this document, the root element may have other declarations, for example namespacing, per normal XML guidelines.

A.1.2.1 Attribute *version*

The only required attribute within an MDDL instance document is the attribute *version* of the root element *mddl*. The *version* is required to explicitly define with which version of MDDL the instance document is conformant. An MDDL version will generally be comprised of major and minor version numbers and possibly some text. *Senders* of MDDL should specify the lowest *version* of MDDL to which the document is conformant. *Recipients* should parse the *version* to determine if internal processing can support the instance document. Subsequent revisions of MDDL will always employ a lexically increasing *version* to facilitate this processing test. Backwards compatibility with each revision of MDDL will be maintained for minor revisions while those modifications that are not backwards compatible will necessitate a major revision. These exceptions will be noted in the appropriate revision statement accompanying each release.

A.1.3 Extension Element *other*

XML is, by its nature, extensible using standard namespace techniques. MDDL seeks to highlight extensions added by the *sender* to give the *recipient* clear indication of such modifications. All extensions to MDDL should be included in the element *other*, which is provided as a child to all *construct*, *container*, *property*, and *product* elements.

When providing extensions, the added elements should include a namespaced element provided by the *sender*. The corresponding namespace declaration should be included on the root element. Each redistributor of an MDDL document may add proprietary elements so a single MDDL instance document may have extensions from multiple *senders*. Thus, clear identification of the source of an extension is compulsory.

A.1.4 Optional Element *header*

The element **header** holds identifying information about the MDDL instance document including identification of the **source** of the document as well as the date and time (**dataDateTime**) when the document was generated. In the current version of MDDL, the content of the **source** is entirely at the discretion of the *sender* and may include other namespaced elements in the extension **other** as desired by the *sender*.

It is encouraged that every MDDL instance document should have a **header** with a **dataDateTime** and **source** for uniform reporting and conveyance of administrative information to the recipient. However, the **header** element and its children are optional. The **header** element, if included, must be the first child element of the root element.

Example of header

```
<mddl version="1.0-final"
  xmlns="http://www.mddl.org/mddl/2001/1.0-final" >
  <header>
    <dataDateTime>2001-11-02T15:00:00Z</dataDateTime>
    <source>Your Data Provider</source>
  </header>
  ... other MDDL content ...
</mddl>
```

The **header** element with recommended **dataDateTime** and **source** elements should be provided as the first child element of the root element.

NOTE: The element **header** is optional but is highly recommended.

Example of header

```
<mddl version="1.0-final"
  xmlns="http://www.mddl.org/mddl/2001/1.0-final"
  xmlns:ypd="http://www.ypd.net/namespace">
  <header>
    <dataDateTime>2001-11-02T15:00:00Z</dataDateTime>
    <source>
      <mdString>Your Data Provider</mdString>
      <other>
        <ypd:slug>Providing you a world of data...</ypd:slug>
        <ypd:location>Anywhere on the globe</ypd:slug>
      </other>
    </source>
  </header>
  ... other MDDL content ...
</mddl>
```

The **source** element extended with *sender* specific extensions in element **other**.

NOTE: All extensions should be properly namespaced.

A.1.5 Standard Attributes

As mentioned previously, MDDL is intentionally light on the use of attributes. There are, however, a few standard attributes that are part of the specification. The **scheme** attribute applies to all *controlled vocabulary properties*. The **xml:lang** attribute is applicable only to the **mdString** element. Attribute **id** can be applied to any element except the root element.

A.1.5.1 Attribute *scheme*

All *controlled vocabularies* enumerate the legal values for that vocabulary in a **scheme** identified by the *sender*. MDDL maintains a default list of *controlled vocabularies* and their **scheme** names on the mddl.org website. A *sender* may use an existing scheme or define its own scheme and make it available to its *recipients*.

Each *property* that should contain a value from a *controlled vocabulary* has a **scheme** attribute on the element definition in the Schema/DTD as well as a default value for that attribute. If a *sender* wishes to specify a value for that *property* that is not from the default *controlled vocabulary*, the **scheme** attribute is included and set to some URI (preferably a URL pointing to the *sender's controlled vocabulary*) that clearly identifies the *sender's* scheme for that *property*.

Note that Schema and DTD validators will not process the value and verify it against the **scheme** specified. If a *recipient* wishes to validate the value and **scheme**, additional processing may be necessary by the *recipient*. If the *sender* does not provide a valid URL with an MDDL conformant *controlled vocabulary* then *sender* specific processing will be necessary.

Example of scheme attribute – not required for default
<code><changeType>previousClose</changeType></code>
The changeType element refers to a default vocabulary and specifies a value from that list. In this case, the default is defined by the Schema and in the registry on the mddl.org website.

NOTE: See mddl.org for a complete list of MDDL provided *controlled vocabularies*.

Example of scheme attribute – <i>sender</i> supplied scheme
<code><changeType scheme=http://www.ydp.net/changeType.xml>meanValue</changeType></code>
The changeType element refers to a <i>controlled vocabulary</i> maintained by the <i>sender</i> . The value “meanValue” is a legal value in that list.

NOTE: *Controlled vocabularies* need not be included in a file directly available to the general public but *senders* are encouraged to make their *controlled vocabularies* readily available on their own website or by registering them with mddl.org.

A.1.5.2 Attribute **xml:lang** and Element **mdString**

All MDDL string *properties* are, by default, specified in United States English. In the event another language is needed, the **mdString** element can be augmented by the standard **xml:lang** attribute. Note that this convention prohibits using the shorthand for *properties* as the **mdString** element must be provided. In addition, multiple language equivalents may be provided given that all non-English equivalents are labeled with the appropriate language per **xml:lang**.

Example of xml:lang attribute

```
<objective>
  <mdString>The objective is ...</mdString>
  <mdString xml:lang="es">El objetivo es...</mdString>
</objective>
```

This contrived example shows how the **xml:lang** attribute can be used to indicate that an **mdString** value is not English. The above example implies that the non-English value “El objetivo es” is an equivalent of the English value “The objective is” in this context.

NOTE: The attribute **xml:lang** can be added to the **mdString** element as needed.

It is imperative that those elements with a **scheme** attribute (denoting a *controlled vocabulary*) use an existing scheme and term from that scheme if it is available. *Controlled vocabularies* are to be processed by computers (not humans) and, as such, providing a vocabulary term in an alternate language does not facilitate processing by the *recipient* nor does it add value to the terminology. A *sender* may specify its own **scheme** for a *controlled vocabulary* that does contain words in a language other than English but should do so only when the default – or other existing – vocabulary does not contain an appropriate representative entry. If a foreign language vocabulary is defined, it is appropriate to provide the **xml:lang** attribute on the element but this value will be ignored when validating the content.

If an existing *controlled vocabulary* contains the appropriate “short” word but a non-English equivalent of the “full” definition is desired, contact the owner of the scheme and request the additional **xml:lang** augmented “full” definitions be added to the scheme. In this way, standardized interoperability is preserved. *Senders* are encouraged to create multiple language *controlled vocabularies* in this same manner. The appendix has an example of a *controlled vocabulary* with multiple languages.

A.1.5.3 Attribute id

As with most XML specifications, each MDDL element can be labeled with the **id** attribute so it may be referenced in XLink, XPointer, or other related referencing mechanisms. MDDL provides this standard identification method for every element except the root element.

Example of id attribute

```
<instrumentIdentifier id="ydp-BSE-MF128">
  <name>Some Mutual Fund</name>
  <code scheme="http://www.ydp.net/XML/scheme/ydpSymbols.xml">
    <mdString>BSE-MF128</mdString>
    <nameRef>../../name</nameRef>
  </code>
</instrumentIdentifier>
```

```
<ref:instrumentIdentifier xlink:ref="ydp-BSE-MF128"/>
```

The first snippet shows the use of the **id** attribute on the **instrumentIdentifier** which is then referenced in the second example.

NOTE: Every element, except the root element, can be identified with the **id** attribute.

A.1.6 Use of element *instrumentIdentifier*

The **instrumentIdentifier** is used to uniquely identify the financial instrument or entity for which the applicable MDDL fragment applies. There may be multiple identifiers relevant at any time and each is identified by a separate **code**. Each **code** has an associated **scheme** and possibly a **rank**. The **scheme** identifies the “owner” of the **code**, or the agency or organization that defines the identifier. The **rank** is a subjective indication by the data provider the relative priority of the **code** and its value in identifying the instrument.

In general, there are four basic types of identifiers in common use:

- “ISIN” (ISO : International Securities Identification Number)
- Identifiers provided by national numbering agencies
- Those provided by special interests or concerns
- The “ticker” symbol used by the exchange (for trading)

In the abstract, the first two are related and subsets of the third and are handled similarly within MDDL. The general form is as follows:

General form of instrumentIdentifier used by special interests or concerns
<pre><instrumentIdentifier> <name>Some Special/Private/Non-ticker Instrument</name> <code scheme="the special scheme name"> <mdString>privatesymbol</mdString> <nameRef>../../name</nameRef> </code> </instrumentIdentifier></pre>
This is the general form of the instrumentIdentifier as used by numbering agencies, private corporations, as well as ISIN. The scheme is assigned a value as identified by the agency or corporation (see the known list of instrumentIdentifier schemes at www.mddl.org). The name element takes on whatever value is recorded by that agency. The mdString element holds the code provided by the referenced agency.

NOTE: The data represented here is for illustration purposes ONLY.

The ticker symbol, when coupled with the Market Identifier Code (ISO 10383) to identify the exchange of listing, clearly identifies the instrument in question (particularly from a trading perspective). This is represented as follows:

General form of instrumentIdentifier for ticker symbols
<pre><instrumentIdentifier> <name>Some Ticker Symbol Instrument</name> <code scheme="http://www.mddl.org/ext/scheme/ticker?mic=miccode"> <mdString>tickersymbol</mdString> <nameRef>../../name</nameRef> </code> </instrumentIdentifier></pre>
This is the form of the instrumentIdentifier as used for ticker symbols of markets that have a MIC code (<i>or should have if missing from current list</i>). The <i>miccode</i> is given the value from the list of Market Identifier Codes (ISO 10383). The <i>tickersymbol</i> is the official symbol for the referenced instrument within the market identified.

NOTE: The data represented here is for illustration purposes ONLY.

When multiple **instrumentIdentifier code** elements are prescribed, one instance should be marked with a rank of “1” to indicate the primary code from the *provider* point of view. The following is a complete example using multiple identifiers.

Example of instrumentIdentifier (multiple code elements)

```
<instrumentIdentifier>
  <name id="name2">Your Data Provider, Incorporated.</name>
  <code scheme="http://www.mddl.org/ext/scheme/iso6166">
    <mdString>012345678987</mdString>
    <nameRef>../../name[id="name2"]</nameRef>
  </code>
  <name id="name1">Your Data Provider, Inc.</name>
  <code scheme="http://www.mddl.org/ext/scheme/ticker?mic=XNYS">
    <mdString>YDP</mdString>
    <nameRef>../../name[id="name1"]</nameRef>
    <rank>1</rank>
  </code>
</instrumentIdentifier>
```

The above example shows an **instrumentIdentifier** with multiple codes from disparate sources. The 2nd **code** (given **rank** 1) is a ticker symbol while the 1st **code** is representative of an ISIN.

Please note that the references to the New York Stock Exchange (MIC:XNYS) and the International Securities Identification Number (ISO 6166) are to show the relationship of the data elements and are not representative of actual corporations nor indicative of relationships with either of those institutions.

NOTE: The data represented here is for illustration purposes ONLY.

A.1.7 Special Controlled Vocabulary *indicatorsType*

Each of the *controlled vocabularies* specified and maintained by MDDL has its specific function but the *indicatorsType controlled vocabulary* bears special discussion because of the content of the vocabulary. The *indicatorsType* has specific indicators relative to various aspects of market data and is defined in such a way that multiple indicators may be specified on each *property* (or more likely, each *container*). For example, one group of indicators differentiates between “realtime” and “delayed” while others specify “actual”, “calculated”, or “estimated”. Indeed, it would seem conceivable that there would be a *property* that is both “delayed” and “calculated”.

While it might be reasonable to split the different groupings of the *indicatorsType* into multiple *controlled vocabularies*, none of the values of this vocabulary affect the values or the relationship of the *properties*. All indicators provide information about the interpretation, reliability of the data provided, or some threshold set by the data. Incomplete processing of indicators will not affect the collection of correct values but may result in the loss of certain nuances associated with the data.

A.2 Base Types

Every *property* within MDDL intended to carry data values is given a *base type*. This type restricts the values that element may contain and instructs the *recipient* how to process and store the data. The *base types* are specified in the Glossary SAF for each *property* and are detailed below.

A.2.1 Base Type *Boolean*

The *base type Boolean* is defined by the XML Schema type `xsd:boolean` and is constrained by that definition. Most *properties* of type Boolean are represented within MDDL by the element `mdBoolean`.

Example of <code>mdBoolean</code>
<code><mdBoolean>true</mdBoolean></code>
<code>mdBoolean</code> may contain any legal value as defined by the Schema type <code>xsd:boolean</code> .

NOTE: Use “true” and “false” for values of `mdBoolean`.

A.2.2 Base Type *DateTime*

The *base type DateTime* is defined by the union of the XML Schema types `xsd:date`, `xsd:dateTime`, and `xsd:time` and is constrained by those definitions. All *properties* of type DateTime are represented within MDDL by the element `mdDateTime`.

Examples of <code>mdDateTime</code>
<code><mdDateTime>2001-11-02</mdDateTime></code>
<code><mdDateTime>2001-11-02T16:20:21Z</mdDateTime></code>
<code><mdDateTime>T16:20:21+02:00</mdDateTime></code>
<code>mdDateTime</code> may contain any legal value as defined by the Schema types <code>xsd:date</code> , <code>xsd:dateTime</code> , or <code>xsd:time</code> .

NOTE: All `mdDateTime` time values must indicate the timezone or offset per `xsd:time`.

A.2.3 Base Type *Decimal*

The *base type Decimal* is defined by the XML Schema type `xsd:decimal` and is constrained by that definition. All *properties* of type Decimal are represented within MDDL by the element `mdDecimal`.

Example of <code>mdDecimal</code>
<code><mdDecimal>789</mdDecimal></code>
<code><mdDecimal>1.23</mdDecimal></code>
<code><mdDecimal>1.37e2</mdDecimal></code>
<code>mdDecimal</code> may contain any legal value as defined by the Schema type <code>xsd:decimal</code> .

NOTE: `mdDecimal` is the only container for numeric content.

A.2.4 Base Type *Duration*

The *base type Duration* is defined by the XML Schema type `xsd:duration` and is constrained by that definition. Most *properties* of type Duration are represented within MDDL by the element `mdDuration`.

Example of mdDuration

```
<mdDuration>P1Y3DT6H</mdDuration>
```

mdDuration may contain any legal value as defined by the Schema type **xsd:duration**. This example specifies a period of 1 year, 3 days, and 6 hours.

NOTE: **mdDuration** implies a range of days or time and not a specific starting or ending date or time.

A.2.5 Base Type String

The *base type String* is defined by the XML Schema type **xsd:string** and is constrained by that definition. All *properties* of type String are represented within MDDL by the element **mdString**.

Example of mdString

```
<mdString>house</mdString>
```

```
<mdString xml:lang="es">casa</mdString>
```

mdString is the container for all text or string based content. Multiple **xml:lang** augmented **mdString** elements can be provided assuming no two **mdString** elements have the same value for **xml:lang**. In no language is specified, U.S. English is assumed.

NOTE: **mdString** holds all string content in English or any language.

A.2.6 Base Type Enumeration

The *base type Enumeration* is a subset of the *base type String*. All *properties* of type *Enumeration* are represented within MDDL by the element **mdString** and include a special **scheme** attribute to reference codes from *controlled vocabularies*.

Example of base type Scheme (currency)

```
<currency>USD</currency>
```

```
<currency scheme="http://www.mddl.org/ext/scheme/iso4217-alpha-3.xml">
<mdString>USD</mdString></currency>
```

A *Enumeration* is implemented as an **mdString**. The first example shows the normal shorthand for representing items from *controlled vocabularies* (in this case one where a default **scheme** is defined). The second shows the equivalent fully expanded.

NOTE: An *enumeration* is an **mdString** that holds codes from *controlled vocabularies*.

A.2.7 Base Type URI

The *base type URI* is defined by the XML Schema type **xsd:anyURI** and is constrained by that definition. All *properties* of type URI are represented within MDDL by the element **mdURI**.

Example of mdURI

```
<mdURI>../../name</mdURI>
```

Any valid URI may be placed inside the element **mdURI** including XLink or XPointer configurations necessary to refer to material in a **references** section.

NOTE: The element **mdURI** is often used to point to referenced material.

A.2.8 Adding Types

It is not recommended that any other *base types* be added outside the formal specification review process. These *base types* are fundamental to the *recipient's* understanding and processing of the data. All requests for modifications or new *base types* should be directed to the MDDL sponsoring agency.

A.3 Classifications

Each *property* within MDDL intended to hold standalone data is defined by a *classification*. The *classification* clarifies the content the *property* holds and identifies a predetermined set of *properties* that may augment that content. The *classification* is defined in the Glossary SAF for each *property*.

A.3.1 Classification Amount

A *property* of *classification Amount* holds a number indicating a quantity of money for a transaction of more than one item. Often, the value of an *Amount property* is the value of some *property* of *classification Price* multiplied by the value of another *property* of *classification Quantity*.

Amount properties can be augmented by the *properties*:

- **calculationType**
- **currency**
- **dataDateTime**
- **fraction**
- **indicatorsType**
- **marketCenter**
- **multiplier**
- **previousDateTime**
- **sequence**
- **source**
- **valuationType**
- **valuationRef**

A.3.2 Classification DateTime

Properties of *classification DateTime* hold a date, time, or date and time as defined by the **mdDateTime** element.

DateTime properties can be augmented by the *properties*:

- **indicatorsType**
- **marketCenter**
- **sequence**
- **source**

A.3.3 Classification Fee

The *classification Fee* denotes a *property* that holds a quantity of money for the cost of a transaction or exchange of goods.

Fee properties can be augmented by the *properties*:

- **calculationType**
- **currency**
- **dataDateTime**
- **fraction**
- **indicatorsType**
- **marketCenter**
- **multiplier**

- **previousDateTime**
- **sequence**
- **source**
- **valuationType**
- **valuationRef**

A.3.4 Classification Measure

A *property* of *classification Measure* holds a number that represents a non-money based indicator or reference value

Measure properties can be augmented by the *properties*:

- **calculationType**
- **dataDateTime**
- **fraction**
- **indicatorsType**
- **marketCenter**
- **multiplier**
- **previousDateTime**
- **sequence**
- **source**
- **valuationType**
- **valuationRef**

A.3.5 Classification Price

Properties of *classification Price* hold a number indicating a quantity of money for a single item.

Classification properties can be augmented by the *properties*:

- **calculationType**
- **currency**
- **dataDateTime**
- **fraction**
- **indicatorsType**
- **marketCenter**
- **multiplier**
- **previousDateTime**
- **sequence**
- **source**
- **valuationType**
- **valuationRef**

A.3.6 Classification Quantity

A *property* of *classification Quantity* holds a number that represents an accumulator or counting of some entity.

Quantity properties can be augmented by the *properties*:

- **calculationType**
- **dataDateTime**
- **fraction**

- **indicatorsType**
- **marketCenter**
- **multiplier**
- **previousDateTime**
- **sequence**
- **source**
- **valuationType**
- **valuationRef**

A.3.7 Other Classifications

The Glossary specifies other classifications that do not imply the required properties like the above classifications.

A.3.7.1 Classification Container

A *container* does not have any content other than child elements.

A.3.7.2 Classification Enumeration

An element of *classification Enumeration* will hold string content but will also contain a **scheme** attribute to identify the *controlled vocabulary*.

A.3.7.3 Classification Simple

This identifies a *property* that has a *base type* but without explicit modifiers.

A.3.7.4 Absence of a Classification

Only *properties* (and *containers*) will have *classifications*. The root element **mddl**, the element **header** (and its children), all *constructs*, and all *domains*, *classes*, and *subclasses* will not have a *classification*.

A.3.7.5 Adding Classifications

It is not recommended that any other *classifications* be added outside the formal specification review process. The basic *classifications* are fundamental to the *recipient's* understanding and processing of the data. As such, MDDL does not support the addition of *classifications* without modification to the base Schema or DTD. All requests for modifications or addition of *classifications* should be directed to the sponsoring agency.

A.4 Constructs

The MDDL concepts of *products* and *properties* permit a *sender* and *recipient* to exchange market data about specific financial instruments in a commonly understood fashion. However, the definition of a *product* does not convey the reason for the exchange of data nor the context in which the data is being provided. This is handled at an upper level in the specification utilizing the concept of *constructs*.

A *construct* is essentially a container for holding one or many *products* with an indication, from the name or hierarchy of the *construct*, as to why the data is being provided or in what context the data should be interpreted. Initially, MDDL provides for two *constructs* – *snap* and *timeseries*.

An MDDL document may contain any number of *constructs*. A *construct* may only contain *products*, *properties*, or some *containers* but may not contain other *constructs*.

A.4.1 Construct *snap*

The *snap construct* holds any number of *products* with data that is valid at the time of production or is related to a specific market condition. The *snap* should be used to hold all “snapshot” quotes or “end-of-day” data delivered to the *recipient*. *Senders* may mark the *snap* with the appropriate *dataDateTime* (at least a date) or *marketStatusType* to clearly indicate the validity of the data or the context in which the data should be processed. For example, an “end-of-day” *snap* of data about a particular instrument may only indicate the date for which the data is valid and an indicator that the market is closed.

Note that a single MDDL document may contain any number of *snap*s and each *snap* may contain multiple differing *products*.

A.4.2 Construct *timeseries*

The *timeseries construct* holds any number of *events* differentiated by time. Each *event* may hold any number of *products* or *properties* as with the *snap construct*. The use of a *timeseries* is warranted when information about a single instrument is desired over a time range with multiple data points. For example, a *timeseries* should be used to convey the closing price of an instrument for each day in the last month or the highest values in each of the five-minute intervals since market open.

Note that a single MDDL document may contain any number of *timeseries* and each *timeseries* should contain multiple *events*.

A.4.3 Adding Constructs

The mechanisms provided in MDDL do not currently allow the addition of *constructs* to MDDL instance documents. Extensibility for *constructs* is currently being discussed by the Technical Committee for implementation in a later version of the specification. Urgent needs for an additional *construct* should be discussed with the sponsor or Technical Committee for guidelines on the plans for this capability.

A.5 Products

MDDL *products* are defined to encapsulate that collection of data needed to define a financial instrument, indicator, or corporate action within a *construct*. The MDDL organization defines *products* and maintains them but individual *senders* may also define proprietary *products* not addressed by the consortium or outside the role of market data as defined by the MDDL sponsor (see “Adding Products” below).

A.5.1 Product Groups as Containers

MDDL *products* are named consistent with the Domains, Classes, and Subclasses identified by the Vocabulary Committee. The element names are postfixed with the designation of Domain, Class, or Subclass as appropriate for convenience to the human reader. For example, the Equity Domain identified by the Vocabulary Committee is given an XML element name of **equityDomain**. It is not intended that these names be parsed because the names of the elements are opaque. All MDDL *products* are simply *containers* for *properties* or other *products*.

For each MDDL element defined as a *product*, an additional *container* element is defined postfixed with the designation Group. For example, the **equityDomain** element has a corresponding **equityDomainGroup**. The Group is only a child of its respective *product*. The Group may contain as its child any of the content the *product* may contain as well as the Group itself.

The Group container is used as a holder of *properties* that are common to all of its children. A product may contain multiple Groups where each Group defines a different set of common *properties* to be inherited by all of its descendants. For example, an **equityDomain** *product* in an instance may contain two **equityDomainGroup** child elements where one defines a currency of Euros and the other defines a currency of Japanese Yen.

A.5.2 Adding Products

The extension element **other** of the main **constructs** can be used as a *container* for any namespaced *products* that the MDDL *sender* would like to add. It is recommended that these extensions match the style and spirit of the other MDDL *products*. Those who provide extensions are encouraged to discuss these with the MDDL organization for inclusion in future versions of the specification. Widespread adoption and use of *products* increases the validity and value of this interchange specification.

A.5.3 List of Products – MDDL 1.0-final

The most comprehensive list of currently supported MDDL *products* developed by the MDDL organization can be found at mddl.org. *Products* developed by the MDDL organization fall into three main categories as defined by the Vocabulary Committee: 1) Instruments & Asset Classes 2) Indicators and 3) Corporate Actions & Events. These *products* and their subdivisions are defined in detail in the Implementation Overview document as well as at mddl.org.

The most comprehensive list of MDDL *products* and documentation can be found at mddl.org. This list is subject to change in subsequent versions.

annuityClass	insuranceSubclassGroup
annuityClassGroup	limitedSubclass
certificateClass	limitedSubclassGroup
certificateClassGroup	moneyMarketSubclass
civDomain	moneyMarketSubclassGroup
civDomainGroup	mutualSubclass
commonClass	mutualSubclassGroup
commonClassGroup	partnershipClass
depositoryClass	partnershipClassGroup
depositoryClassGroup	preferredClass
equityDomain	preferredClassGroup
equityDomainGroup	sharedSubclass
exchangeTradedSubclass	sharedSubclassGroup
exchangeTradedSubclassGroup	trustClass
fundClass	trustClassGroup
fundClassGroup	umbrellaSubclass
indexDomain	umbrellaSubclassGroup
indexDomainGroup	unitsSubclass
insuranceSubclass	unitsSubclassGroup

A.6 Properties

All elements below the *construct* or *product* delineation are considered *properties* whether they are descriptors or modifiers of those descriptors. Each *property* is assigned a *base type* (or no type) as well as a *classification* to clue the *recipient* as to how the data should be processed or stored.

There are very clear guidelines governing the use of *properties* and the inheritance of *properties* through the element hierarchy defined as part of the MDDL specification. A *property* applies to its siblings as well as its parent and is inherited by the descendants of all siblings unless specifically overridden by another *property* of the same name.

The *classification* of a *property* identifies which *properties* are necessary to fully specify that element of data. A *recipient* should collect all *properties* related to an element of a particular *classification*. If these *properties* are not children of the element of interest, they may be inherited from elements higher up the hierarchy. To find a necessary *property*, check the children (not all the descendants) of the element of interest and the siblings of all ancestors. Do not scan the descendants of the siblings of the ancestors. Once all relevant *properties* are assembled (based on the *classification*), the *property* of interest is fully described.

Example of *property* inheritance

```
<equityDomain>
  <equityDomainGroup>
    <currency>USD</currency>
    ... other MDDL elements with currency USD ...
  </equityDomainGroup>
  <equityDomainGroup>
    <currency>JPY</currency>
    ... other MDDL elements with currency JPY ...
  </equityDomainGroup>
</equityDomain>
```

Use of the Group *container* permits segregation of descendant elements of differing currencies. The *property* inheritance rules specify that *currency* applies to its *container*, the siblings, and all descendants of the siblings unless explicitly overridden by another *currency*.

NOTE: A *container* provides the facility to group common *properties*.

Example of *property* shorthand

```
<administrator>
  <mdString>Julie Frank</mdString>
</administrator>
...
<administrator>Julie Frank</administrator>
```

The two instances of the element *administrator* in the above example are functionally equivalent given that no additional *properties* were necessary to modify the *administrator* element beyond the English content *mdString* designator.

NOTE: When the *base type* element is the only child element within a *property* element, the *base type* element can be removed leaving the content of the *base type* element as the content of its parent *property*.

Example of *properties* – shorthand and inheritance

```
<trade>
  <last>
    <mdDecimal>127.32</mdDecimal>
    <currency>JPY</currency>
  </last>
</trade>
```

```
<trade>
  <currency>JPY</currency>
  <last>127.32</last>
</trade>
```

Due to the use of the *property* shorthand and the inheritance from an ancestor the implication for both examples is the same – the **last trade** is 127.32 Japanese Yen.

NOTE: Inheritance can simplify the specification of some *properties*.

Most *properties* are associated with a particular piece of market data and have a very specific definition in context. There are a few *properties* that have been added to the MDDL specification that are applicable to all other elements.

A.6.1 Common Property *fraction*

MDDL is intended to convey market data for processing by computer systems and, as such, most of the numeric content is conveyed in *properties* of the *base type Decimal*. However, some of this data will be displayed to humans and, additionally, some of this data is provided, or intended for display, in fractions. In the event that a number provided as a decimal originated as a fraction or is intended to be displayed as a fraction, the *property fraction* can be included to imply this fact and denote the denominator of the originating fraction.

The **fraction** element is devoid of content but has attributes **actualDenominator** and **displayDenominator**. Note that it is not necessary to provide the numerator when specifying a fraction because the numerator can be derived from multiplying the **actualDenominator** with the non-integer portion of the provided decimal. This mathematical product should always yield, when properly rounded, a whole integer thus permitting the *recipient* to reconstruct the numerator and determine the magnitude of rounding, if so desired.

A.6.1.1 Attribute **actualDenominator**

The **actualDenominator** attribute of the **fraction** element denotes the original denominator of the number contained in a *property* that is represented in decimal format. By comparing the denominator represented with the actual decimal number, a processor can (if necessary) determine the rounding that was performed to convert the original fraction to decimal format.

A.6.1.2 Attribute **displayDenominator**

When numeric decimal content is to be displayed to the end-user as a fraction by convention, the attribute **displayDenominator** conveys the *sender's* understanding of the denominator that should be used to display the fraction. The processor displaying the fraction should round the decimal number to the nearest numerator per the constraints defined by the **displayDenominator**.

Example of fraction – no rounding

```
<last>
  <mdDecimal>17.15625</mdDecimal>
  <fraction actualDenominator="32" displayDenominator="8">
</last>
```

The decimal value of **last** is 17.15625 and there is no rounding (.15625 is an integer number of 32nds in that $.15625 = 5/32$). However, the *sender* has indicated that the number should be displayed in 8ths. Therefore, the number, as indicated by the *sender*, should be rounded to display as “17 1/8” (or whatever format is expected for fractions in the user’s environment).

NOTE: The element **fraction** contains no child elements.

Example of fraction – actualDenominator less than displayDenominator

```
<last>
  <mdDecimal>29.25</mdDecimal>
  <fraction actualDenominator="4" displayDenominator="8">
</last>
```

The decimal value of **last** is 29.25 and there is no rounding (.25 is an integer number of 4ths in that $0.25 = 1/4$). However, the *sender* has indicated that the number should be displayed in 8ths. Therefore, the number, should be converted to display as “29 2/8” (or whatever format is expected for fractions in the user’s environment).

NOTE: The **actualDenominator** can be less than the **displayDenominator**.

Example of fraction – rounding

```
<last>
  <mdDecimal>29.33333</mdDecimal>
  <fraction actualDenominator="6" displayDenominator="3">
</last>
```

The decimal value of **last** is 29.33333 and there is rounding (.33333 is not an integer number of 6ths). The *recipient* can assume that some rounding has taken place because $.3333 \times 6$ is not an integer. Additionally, the *sender* has indicated that the number should be displayed in 3rds. Therefore, the number, as indicated by the *sender*, should be displayed as “29 1/3” (or whatever format is expected for fractions in the user’s environment).

NOTE: It may be acceptable to display numbers in decimal even though a **fraction** is provided.

A.6.2 Common Property multiplier

In some instances it may be convenient to provide numeric content (*properties* of *base type Decimal*) at a different order of magnitude or scale. The element **multiplier** can be used to provide a multiplication factor for all sibling *properties* and their descendants. For example, many of the quotes and trades of British stocks are processed in pence even though the base currency is Great Britain Pounds (GBP). A **multiplier** with a value of “.01” attached to the hierarchy can clue the *recipient* that the values are in pence even though the currency is pounds.

Example of multiplier

```
<currency>
  <mdString>GBP</mdString>
  <multiplier>.01</multiplier>
</currency>
```

The multiplier attached to this currency indicates that the prices affected by this currency (through *property* inheritance) should be multiplied by .01 to match the currency.

NOTE: The **multiplier** element should not be used indiscriminately. Misuse of this *property* can cause undue processing by the *recipient*.

A.6.3 Common Property *period*

The element **period** permits the specification of a range of time that includes a duration starting at a specific date or time and/or ending at a specific date or time. In addition, it is possible to specify a recurring range of time between specific start and end dates or times. In general, **duration** must always be specified while **start**, **end**, and **recurring** are optional depending on the intention. For example, if **start** and **end** are both specified and the intent is to specify an absolute start and end date or time, **duration** may be omitted. If however, the primary intent is to provide a range of time but also indicate absolutely when that duration began and ended, then **duration** should be provided.

A.6.3.1 Element *start*

The **start** defines the starting date and/or time of the **period** and has content equivalent to the **mdDateTime** element. If **duration** specifies hours, minutes, or seconds then **start** should specify a time. Additionally, **start** may specify a date on which the period begins.

A.6.3.2 Element *end*

This element defines the ending date and/or time of the **period**. This element has content equivalent to the **mdDateTime** element. If a **duration** is specified that includes hours, minutes, or seconds then **end** should specify a time. Additionally, **end** may specify a date on which the period ends.

A.6.3.3 Element *duration*

The duration of the **period** indicates the length of time for which the **period** is valid and has content equivalent to the **mdDuration** element. The value of **duration** has implications on the appropriate values of **start** and **end**. If **duration** specifies a number of hours, minutes, or seconds then **start** and/or **end** should also specify a time (rather than a date). If **duration** specifies a number of years, months, or days (but not hours, minutes, or seconds) then **start** and/or **end** should specify a date (rather than a time). If **duration** specifies time and days then **start** and/or **end** should specify both a date and a time.

Example of period with duration

```
<period>
  <duration>P1Y3M6D</duration>
</period>
```

A period of 1 year, 3 months, and 6 days as defined by the rules of **mdDuration**.

NOTE: A **duration** of a **period** has the same content model as **mdDuration**.

Example of period with duration and start

```
<period>
  <start>2001-11-02</start>
  <duration>P1Y3M6D</duration>
</period>
```

A day period of 1 year, 3 months, and 6 days beginning on 02 November 2001.

NOTE: A **duration** may begin at a particular **start** date or time.

Example of period with duration and end

```
<period>
  <duration>P2H30M</duration>
  <end>T03:30:00Z</end>
</period>
```

A time period of 2 hours 30 minutes beginning at 03:30 a.m. GMT (Zulu).

NOTE: A **duration** may terminate at a specific **end** date or time.

Example of period with duration, start, and end

```
<period>
  <start>2001-11-01T12:30:00Z</start>
  <duration>P1D1H45M</duration>
  <end>2001-11-02:14:15:00Z</end>
</period>
```

A period of 1 day, 1 hour, and 45 minutes beginning at the absolute time of 12:30:00 GMT on 01 November 2001 and ending at 14:15:00 GMT on 02 November 2001.

NOTE: If **duration** is provided with **start** and **end**, the **duration** is most important.

A.6.3.4 Element *recurring*

This element indicates whether the period is recurring between the start and end date or times. This element is of *base type Boolean*.

Example of period with duration, start, end, and recurring

```
<period>
  <start>2001-11-01T11:00:00Z</start>
  <duration>P1H20M</duration>
  <end>2001-11-02T14:15:00Z</end>
  <recurring>true</recurring>
</period>
```

The above implies an event that happens every 1 hour and 20 minutes beginning at 11:00:00 GMT on 01 November 2001 and ending on or before 14:15:00 GMT on 02 November 2001.

NOTE: A **recurring** event should specify a **start** and/or **end** date and/or time consistent with the granularity of the specified **duration**.

A.6.4 Common Property *sequence*

This element allows the *sender* to identify a particular MDDL instance document uniquely so that it may be referenced from another document. A *sender*, when specifying sequences, should always use a lexically increasing designator. The element **sequence** may be specified at the root element, *construct*, or *product* levels.

NOTE: The exact format and ramifications of this element and its child element, **replaces**, are still under discussion by the Technical Committee.

A.6.4.1 Element *replaces*

When a *sender* wishes to modify or correct the contents of a previously published MDDL instance document that contains a *sender* provided **sequence**, the *sender* uses the **replaces** element with a new **sequence** identifier. The value of the **replaces** element should match exactly the value of the **sequence** element of a previously published document.

A.6.5 Adding Properties

As with *products*, the extension element **other**, available on all existing *products* and *properties* can be used as a *container* for any *properties* that the MDDL *sender* would like to add. It is recommended that these extensions match the style and spirit of the other MDDL *properties*. Those who utilize extensions are encouraged to discuss these with the MDDL organization for inclusion in future versions of the specification. Widespread adoption and use of *properties* increases the validity and value of this interchange specification.

A.6.6 List of Properties – MDDL 1.0-final

The most comprehensive list of MDDL *properties* and applicable documentation can be found at mddl.org. This list is subject to change in subsequent versions.

administrator	high	previousClose
advisor	indexBaseDate	priceRef
ask	indexBaseValue	purchasingMinimums
backLoad	indexMethodology	qualifiedPlans
baseValue	indexMultiplier	rank
benchmarkStrategy	indexObjective	recurring
bid	indexWeighting	replaces
calculationType	indicatorsType	restrictionsType
capitalization	industry	revenues
capitalizationType	instrumentIdentifier	role
change	instrumentStatusType	sequence
changeDirection	investingRestrictions	session
changeType	investingStyle	sessionStatusType
close	investingTechniques	settlement
closeType	issuePrice	settlementType
code	last	sharesOutstanding
component	legalAdvisor	sharesOutstandingType
componentIdentifier	lotSize	size
componentMultiplier	low	snap
componentValue	marketCenter	source
currency	marketStatusType	spread
custodian	mdBoolean	start
dataDateTime	mdDateTime	targetPublic
dayOfWeek	mdDecimal	taxType
days	mddl	tickDirection
delayFactor	mdDuration	timeseries
distributionType	mdString	timezone
dividend	mdURI	trade
duration	mid	tradingStatusType
earnings	multiplier	transferAgent
earningsRef	name	transferFees
earningsType	nameRef	turnover
end	nav	underlying
event	numberHolders	valuationType
exchangeConditions	objective	volatility
exchangeIdentifier	objectiveType	volatilityCalculation
exchangeName	open	volatilityType
firstDealing	openType	volume
firstSettlement	orders	volumeType
fraction	paymentFrequency	vwap
frontLoad	peRatio	vwop
fundStrategyType	peRatioType	weighting
header	period	yield

A.7 Reference Content

In many situations, it may be desirable to refer to some data within an MDDL document to avoid duplicating the content. MDDL provides the capability, using the concept of references, to refer to previously published content. This content may be in another MDDL document or may be within the same instance. All referenced content is placed within the **references** element of an MDDL document immediately following the **header** (if it exists).

A.7.1 Referenced Element Trees

Within the **references** element, any *container* or *property* may be included provided that the element is marked with the attribute **id**. The top level element may then contain any subelements as is normal for that element. The **references** section may contain multiple children, each of which is to be referenced by the identifying attribute.

A.7.1.1 Attribute *id*

The attribute **id** may be used on any MDDL element for use with XPointer or XLink mechanisms. It is required on the child elements of the **references** element and contains a unique string within the instance document for later reference.

A.7.2 Referencing Element Trees

Content within the **references** section can be *implicitly* included anywhere within an MDDL document by using the **ref** namespaced elements and the appropriate attributes and subelements.

A.7.1.2 Attribute *xlink:href*

All **ref** namespaced elements have an **xlink:href** attribute that must be specified to indicate which referenced content (see attribute **id**) should be implicitly included at that location in the instance document. This XLink attribute may reference any valid **id** within the current instance or another MDDL document.

A.7.1.3 Attribute *xlink:role*

Each **ref** namespaced element has an **xlink:role** defined to indicate the purpose of the reference content.

A.7.1.4 Attribute *xlink:type*

Every **ref** namespaced element is assigned an **xlink:type** of “simple” per the XLink specification guidelines.

A.7.1.5 Element *delete*

To be documented.

A.7.1.6 Element *replace*

To be documented.

A.7.1.7 Element *addChildrenOf*

To be documented.

Market Data Definition Language (MDDL)

Appendix B – Example MDDL Instance Documents

Version 1.0-final 02 November 2001



<http://www.mddl.org/>

B Example MDDL Instance Documents

This appendix includes example instance documents prepared by the Technical Committee to illustrate the use of the various features of MDDL. The examples seek to demonstrate some of the conventions of MDDL as well as the specific application of some of the *constructs*, *products*, and *properties* that are the foundation of MDDL.

While these documents are valid MDDL, do not infer that these documents represent what is expected from an MDDL *sender* as the examples only show some of the data that would normally be delivered. Instead, note how the *properties* are placed in relation to one another for inheritance and conveyance of the principal concepts.

Additional example XML files are provided showing the format of the *controlled vocabulary* files and Schema Adjunct Framework file for the Glossary.

Example 1: *Construct* **snap** of a Mutual Fund *product*.

Example 2: *Construct* **timeseries** of a Common Equity *product*.

Example 3: *Construct* **snap** of an Index, Mutual Fund, and Common Equity *products* illustrating a use of **references**.

Example 4: *Controlled vocabulary* scheme illustrating the use of multiple languages.

Example 5: Excerpt from the Schema Adjunct Framework file for the Glossary.

B.1 Example 1 – *snap* (Mutual Fund)

The following is a complete example of an MDDL instance document from a fictitious data provider, ypd.net, about a fictitious Spanish Mutual Fund MF128 on the Barcelona Stock Exchange. The Spanish MF128 is derived from unequal shares of the French Common Equity XY12 and the United States Common Equity PQ39.

```
<mddl version="1.0-final"
  xmlns="http://www.mddl.org/mddl/2001/1.0-final">

  <header>
    <dataDateTime>2001-11-02T16:20:21Z</dataDateTime>
    <source>Your Data Provider</source>
  </header>

  <snap>
    <civilDomain>
      <fundClass>
        <mutualSubClass>
          <instrumentIdentifier>
            <name>Some Mutual Fund</name>
            <code
              scheme="http://www.ypd.net/XML/scheme/ypdSymbols.xml">
              <mdString>BSE-MF128</mdString>
              <nameRef>../../name</nameRef>
            </code>
          </instrumentIdentifier>
          <exchangeIdentifier>
            <code
              scheme="http://www.ypd.net/XML/scheme/ypdExchanges.xml"
              >BSE</code>
          </exchangeIdentifier>
          <currency>
            <mdString>ESP</mdString>
          </currency>
          <dataDateTime>2001-11-02</dataDateTime>
          <administrator>Curious George</administrator>
          <firstDealing>1984-08-09</firstDealing>
          <nav>
            <change>
              <mdDecimal>2.10</mdDecimal>
              <changeType>previousClose</changeType>
            </change>
            <last>
              <mdDecimal>549.62</mdDecimal>
              <dataDateTime>15:30:00Z</dataDateTime>
            </last>
          </nav>
          <baseValue>0.0</baseValue>
          <weighting>1.0</weighting>
          <underlying>
            <mdDecimal>273.40</mdDecimal>
            <instrumentIdentifier>
              <name>Some French Company</name>
              <code
                scheme="http://www.ypd.net/XML/scheme/ypdSymbols.xml">
```

```
        <mdString>PSE-XY12</mdString>
        <nameRef>../../../../name</nameRef>
    </code>
</instrumentIdentifier>
<componentValue>27.34</componentValue>
<multiplier>10</multiplier>
</underlying>
<underlying>
    <mdDecimal>276.22</mdDecimal>
    <instrumentIdentifier>
        <name>Some United States Company</name>
        <code
            scheme="http://www.ypd.net/XML/scheme/ypdSymbols.xml">
            <mdString>NYSE-PQ39</mdString>
            <nameRef>../../../../name</nameRef>
        </code>
    </instrumentIdentifier>
    <componentValue>19.73</componentValue>
    <componentMultiplier>14</componentMultiplier>
</underlying>
</mutualSubclass>
</fundClass>
</civDomain>
</snap>
</mddl>
```

B.2 Example 2 – *timeseries* (Common Equity)

The following is a complete **timeseries** example of an MDDL instance document from a fictitious data provider, ypd.net, about a fictitious Hong Kong equity, XC2, on the Hong Kong SE. The example shows four sequential events reporting the last trade.

```
<mddl version="1.0-final"
  xmlns="http://www.mddl.org/mddl/2001/1.0-final">

  <header>
    <dataDateTime>2001-11-02T16:47:04Z</dataDateTime>
    <source>Your Data Provider</source>
  </header>

  <timeseries>
    <instrumentIdentifier>
      <name>Some Hong Kong Company</name>
      <code>
        scheme="http://www.ypd.net/XML/scheme/ypdSymbols.xml">
          <mdString>HKSE-XC2</mdString>
          <nameRef>../../name</nameRef>
        </code>
      </instrumentIdentifier>
      <exchangeIdentifier>
        <code>
          scheme=http://www.ypd.net/XML/scheme/ypdExchanges.xml
          >HKSE</code>
        </exchangeIdentifier>
        <currency>HKD</currency>
        <dataDateTime>2001-11-01</dataDateTime>
        <event>
          <equityDomain><commonClass><trade><last>
            <mdDecimal>188.50</mdDecimal>
            <dataDateTime>2001-11-01T16:20:12Z</dataDateTime>
            </last></trade></commonClass></equityDomain>
          </event>
          <event>
            <equityDomain><commonClass><trade><last>
              <mdDecimal>188.32</mdDecimal>
              <dataDateTime>2001-11-01T16:20:18Z</dataDateTime>
              </last></trade></commonClass></equityDomain>
            </event>
            <event>
              <equityDomain><commonClass><trade><last>
                <mdDecimal>188.76</mdDecimal>
                <dataDateTime>2001-11-01T16:21:03Z</dataDateTime>
                </last></trade></commonClass></equityDomain>
              </event>
              <event>
                <equityDomain><commonClass><trade><last>
                  <mdDecimal>189.04</mdDecimal>
                  <dataDateTime>2001-11-01T16:21:33Z</dataDateTime>
                  </last></trade></commonClass></equityDomain>
                </event>
              </timeseries>
            </mddl>
```

B.3 Example 3 – Basic references (Index)

The following is an example of an MDDL instance document from a fictitious data provider, ypd.net, about a fictitious Swedish index called SWRC. The Index is derived from the Norwegian Common Equity RD77 and the Finnish Mutual Fund SC42.

```
<mddl version="1.0-final"
  xmlns="http://www.mddl.org/mddl/2001/1.0-final">

  <header>
    <dataDateTime>2001-11-02T16:20:21Z</dataDateTime>
    <source>Your Data Provider</source>
    <references>
      <instrumentIdentifier id="ydp-SW-SWRC">
        <name>Some Swedish Index</name>
        <code scheme="http://www.ypd.net/XML/scheme/ypdSymbols.xml">
          <mdString>SW-SWRC</mdString>
          <nameRef>../../../../name</nameRef>
        </code>
      </instrumentIdentifier>
      <instrumentIdentifier id="ydp-NSE-RD77">
        <name>Some Norwegian Company</name>
        <code scheme="http://www.ypd.net/XML/scheme/ypdSymbols.xml">
          <mdString>NSE-RD77</mdString>
          <nameRef>../../../../name</nameRef>
        </code>
      </instrumentIdentifier>
      <instrumentIdentifier id="ydp-HEX-SC42">
        <name>Some Finnish Mutual Fund</name>
        <code scheme="http://www.ypd.net/XML/scheme/ypdSymbols.xml">
          <mdString>HEX-SC42</mdString>
          <nameRef>../../../../name</nameRef>
        </code>
      </instrumentIdentifier>
    </references>
  </header>

  <snap>

    <equityDomain>
      <commonClass>
        <ref:instrumentIdentifier xlink:href="ydp-NSE-RD77">
          <trade>
            <last>
              <mdDecimal>39.17</mdDecimal>
              <dataDateTime>2001-11-02T16:20:18Z</dataDateTime>
            </last>
          </trade>
        </commonClass>
      </equityDomain>

      <civDomain>
        <fundClass>
          <mutualSubclass>
            <ref:instrumentIdentifier xlink:href="ydp-HEX-SC42">
              <nav>
                <change>
```

```
        <mdDecimal>2.10</mdDecimal>
        <changeType>previousClose</changeType>
      </change>
      <last>
        <mdDecimal>27.36</mdDecimal>
        <dataDateTime>2001-11-02T15:30:00Z</dataDateTime>
      </last>
    </nav>
  </mutualSubclass>
</fundClass>
</civDomain>

<indexDomain>

  <ref:instrumentIdentifier xlink:href="ydp-SW-SWRC"/>
  <currency>SWK</currency>
  <dataDateTime>2001-11-02</dataDateTime>

  <indexValue>292849.824768</indexValue>
  <indexBaseDate>2000-10-03</indexBaseDate>
  <indexBaseValue>2700</indexBaseValue>
  <indexMultiplier>3.2</indexMultiplier>
  <indexWeighting>4.72</indexWeighting>

  <baseValue>10.0</baseValue>
  <weighting>3.3</weighting>

  <component>
    <mdDecimal>10115.358</mdDecimal>
    <ref:instrumentIdentifier xlink:href="ydp-NSE-RD77">
    <componentValue>39.17</componentValue>
    <componentMultiplier>78</componentMultiplier>
  </component>

  <component>
    <ref:instrumentIdentifier xlink:href="ydp-HEX-SC42"/>
    <mdDecimal>8429.784</mdDecimal>
    <componentValue>27.36</componentValue>
    <componentMultiplier>93</componentMultiplier>
  </component>

</indexDomain>

</snap>

</mddl>
```

B.4 Example 4 – Controlled Vocabulary Scheme File

The following example scheme file illustrates how an MDDL *controlled vocabulary* can be specified in multiple languages. The value contained in the **short** element is the value given to the *property* referencing this scheme.

```
<?xml version="1.0" encoding="utf-8"?>
<mddlScheme>
  <head>
    <dateTime>2001-11-01T120000Z</dateTime>
    <title>MDDL Days of the Week Scheme</title>
    <element>daysOfWeek</element><parent>days</parent>
    <definition>Days of the Week</definition>
  </head>
  <value><short>M</short><full>Monday</full>
    <full xml:lang="de">Montag</full>
    <full xml:lang="fr">Lundi</full>
    <full xml:lang="it">Lunedì</full>
  </value>
  <value><short>T</short><full>Tuesday</full>
    <full xml:lang="de">Dienstag</full>
    <full xml:lang="fr">Mardi</full>
    <full xml:lang="it">Martedì</full>
  </value>
  <value><short>W</short><full>Wednesday</full>
    <full xml:lang="de">Mittwoch</full>
    <full xml:lang="fr">Mercredi</full>
    <full xml:lang="it">Mercoledì</full>
  </value>
  <value><short>R</short><full>Thursday</full>
    <full xml:lang="de">Donnerstag</full>
    <full xml:lang="fr">Jeudi</full>
    <full xml:lang="it">Giovedì</full>
  </value>
  <value><short>F</short><full>Friday</full>
    <full xml:lang="de">Freitag</full>
    <full xml:lang="fr">Vendredi</full>
    <full xml:lang="it">Venerdì</full>
  </value>
  <value><short>S</short><full>Saturday</full>
    <full xml:lang="de">Samstag</full>
    <full xml:lang="fr">Samedi</full>
    <full xml:lang="it">Sabato</full>
  </value>
  <value><short>U</short><full>Sunday</full>
    <full xml:lang="de">Sonntag</full>
    <full xml:lang="fr">Dimanche</full>
    <full xml:lang="it">Domenica</full>
  </value>
</mddlScheme>
```

B.5 Example 5 – Schema Adjunct Framework Glossary

The below excerpt from the MDDL Glossary illustrates the use of the Schema Adjunct Framework to define enhancing information, not included in the Schema, about the MDDL elements. For example, note the definition of the element **classification** to provide the MDDL *classification* for the *properties*.

```
<?xml version="1.0"?>

<schema-adjunct xmlns:mddl="http://www.mddl.org/mddl/2001/1.0-final">

  <element context="advisor">
    <mddl:classification>SimpleString</mddl:classification>
    <mddl:baseType>String</mddl:baseType>
    <mddl:definition xml:lang="en" locale="us">
      The advisor of record.
    </mddl:definition>
    <mddl:label xml:lang="en" locale="us" length="s">
      >Adv</mddl:label>
    <mddl:label xml:lang="en" locale="us" length="m">
      >Advis</mddl:label>
    <mddl:label xml:lang="en" locale="us" length="l">
      >Advisor</mddl:label>
    </element>

    <element context="close">
      <mddl:classification>Price</mddl:classification>
      <mddl:baseType>Decimal</mddl:baseType>
      <mddl:definition xml:lang="en" locale="us">
        The final price as of market close.
      </mddl:definition>
      <mddl:note>See closeType.</mddl:note>
      <mddl:label xml:lang="en" locale="us" length="s">
        >Cls</mddl:label>
      <mddl:label xml:lang="en" locale="us" length="m">
        >Close</mddl:label>
      <mddl:label xml:lang="en" locale="us" length="l">
        >Close</mddl:label>
      </element>
    </schema-adjunct>
```